

UE Intelligence Artificielle - Polytech Lyon

Partie 3: Apprentissage profond par renforcement

CM6: Deep QNetwork

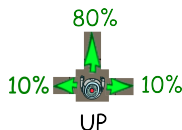
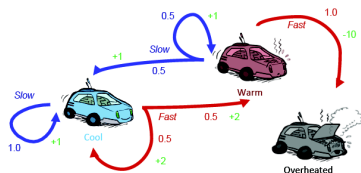
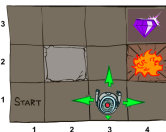
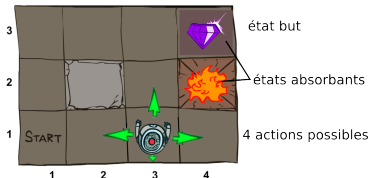
Laëtitia Matignon
laetitia.matignon@univ-lyon1.fr
Equipe SyCoSMA@LIRIS

- 1 Rappels : Apprentissage par renforcement
- 2 Limites des approches classiques
- 3 Deep Q Learning

Processus décisionnel de Markov (MDP)

Modèle MDP $\langle S, A, T, R \rangle$

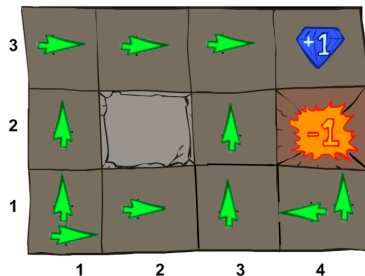
- ensemble fini d'états : S
- ensemble fini d'actions : A ou $A(s)$
- fonction de transition $T : S \times A \times S \rightarrow [0; 1] : T(s, a, s')$ est la probabilité d'atteindre s' lorsque l'on fait a dans s
- fonction de renforcement $R : S \times A \times S \rightarrow \mathbb{R} : \text{récompense}$

fonction de transition T fonction de récompense R

Processus décisionnel de Markov (MDP)

Solution au MDP

Politique notée $\pi : S \rightarrow A$ qui associe une (ou plusieurs) action(s) à chaque état.



exemple de politique

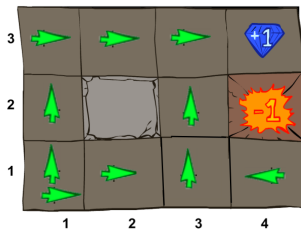
Processus décisionnel de Markov (MDP)

Objectif

Trouver la politique optimale notée π^* , qui donne pour tout état, la ou les action(s) permettant de maximiser les récompenses que l'on espère obtenir à travers la séquence d'états futurs.

π^* maximise la somme des récompenses futures, pondérée par $\gamma \in [0, 1[$:

$$G_t = r_1 + \gamma r_2 + \gamma^2 r_3 + \dots = \sum_{t=0}^{\infty} \gamma^t r_{t+1}$$

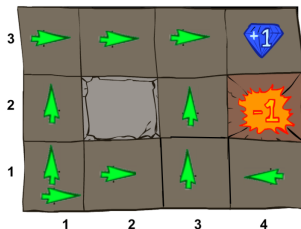


Processus décisionnel de Markov (MDP)

Fonction de valeur

La fonction de valeur $V^\pi(s)$ d'un état s évalue l'espérance de récompenses cumulées G si l'agent démarre dans l'état s et suit la politique π :

$$V^\pi(s) = E_\pi \left\{ \sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_0 = s \right\}$$



0,8100	0,9000	1,0000	
0,7290		0,9000	
0,6561	0,7290	0,8100	0,7290

Fonction de valeur V associée à π dans un environnement déterministe

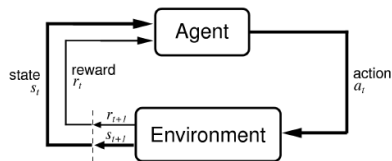
Apprentissage par renforcement

Hypothèses sur le modèle MDP

T et R inconnus de l'agent :

- L'agent ne connaît pas à l'avance les effets de ses actions
- L'agent ne connaît pas à l'avance les actions et/ou états récompensés

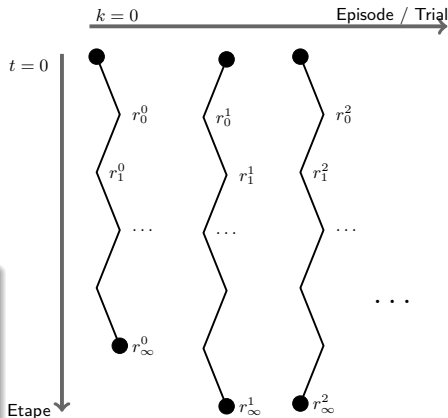
Apprentissage par renforcement



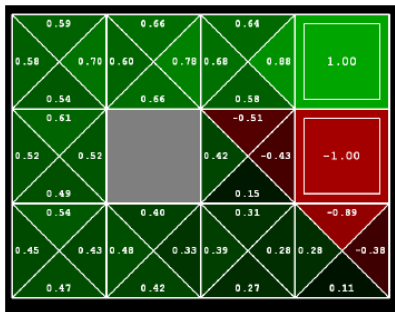
Agent plongé dans un MDP **sans**
connaissance de R et T .

Double temporalité :

- **Étape ou pas** t (une **action** est effectuée)
- **Épisode** k (ensemble d'étapes/actions de l'état initial à un état absorbant)



Apprentissage par renforcement

Fonction de valeur d'action Q

$$Q : S \times A \rightarrow \mathbb{R}$$

- $Q^\pi(s, a)$ évalue le retour espéré lorsque l'on effectue l'action a dans l'état s puis que l'on suit π : $Q^\pi(s, a) = E\{\sum_{t=0}^{\infty} \gamma^t r_{t+1} | \pi, s_t = s, a_t = a\}$
- Liens entre V et Q : $\forall s \in S \quad V(s) = \max_a Q(s, a)$

Amélioration de politique en-ligne

Politique gloutonne : action de plus grande valeur dans s : $\pi(s) = \arg \max_{a \in A} Q(s, a)$

Apprentissage par renforcement

Q-Learning

Initialisation aléatoire de $Q(s, a) \forall s, a$

A chaque étape :

- L'agent est dans l'état s , il choisit l'action a selon une **stratégie d'exploration**

Stratégie d'exploration ϵ -greedy (ou ϵ -glouton)

Explore avec une probabilité ϵ :

$\pi(s) =$

- action d'exploration (aléatoire parmi **toutes** les actions possibles) avec la probabilité ϵ
- action gloutonne ($a = \arg \max_a Q(s, a)$) avec la probabilité $1 - \epsilon$

Apprentissage par renforcement

Q-Learning

Initialisation aléatoire de $Q(s, a) \forall s, a$

A chaque étape :

- L'agent est dans l'état s , il choisit l'action a selon une stratégie d'exploration
- L'agent arrive dans l'état s' et reçoit la récompense r
- L'agent fait une **nouvelle évaluation** de $q(s, a)$ en calculant la somme pondérée des récompenses qu'il peut obtenir depuis s :

$$q(s, a) = \underbrace{r}_{\text{obtenu dans } s} + \gamma \underbrace{\max_{b \in A} Q(s', b)}_{\text{évaluation de la somme des récompenses espérées depuis } s'}$$

- L'agent **met à jour** $Q(s, a)$ selon :

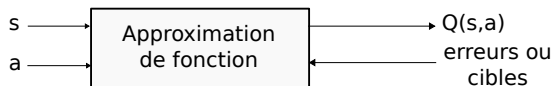
$$Q(s, a) = \underbrace{Q(s, a)}_{\text{ancienne estimation}} + \alpha \times (\underbrace{q(s, a) - Q(s, a)}_{\text{nouvelle} - \text{ancienne estimation} = \text{erreur}})$$

Récap : l'agent dans l'état s fait l'action a et arrive dans s' , reçoit la récompense r :

$$Q(s, a) = (1 - \alpha)Q(s, a) + \alpha \times (r + \gamma \max_{b \in A} Q(s', b))$$

Limites des approches classiques

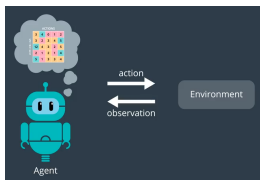
3 façons de représenter la Q fonction :



$Q(s, a) =$

- table : QLearning
- Approximation linéaire et vecteur de *features* expertes : Approximate Q-learning
- Approximation non-linéaire (réseau de neurones) : ce cours !

Approches tabulaires



- utilisation d'une table pour stocker les Q -valeurs
- chaque expérience (s, a, s', r) à l'instant t ne permet d'apprendre que pour le couple état-action (s, a) correspondant à cette expérience.

Problème 1 : espace mémoire

- Besoin de stocker $|S| \times |A|$ éléments.

Problème 2 : Lenteur de l'apprentissage

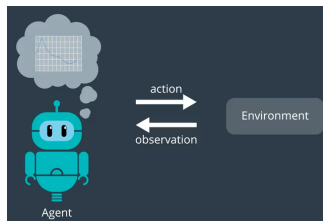
- il faut visiter tous les états un grand nombre de fois
- pas de **généralisation** de l'expérience apprise à des états jamais rencontrés

Problème 3 : Discrétisation

Hypothèse d'espaces d'états **discrets** et finis

- Comment choisir une discrétisation adaptée ?

Fonctions d'approximation



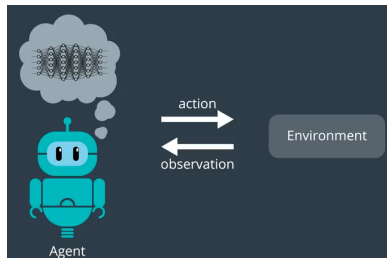
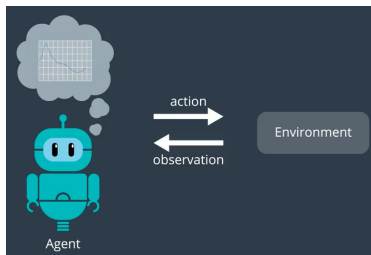
Approximation **linéaire** de la fonction Q

$$Q_w(s, a) = \sum_i w_i f_i(s, a) = w_1 f_1(s, a) + w_2 f_2(s, a) + \dots + w_n f_n(s, a)$$

- n fonctions caractéristiques $f_i : S \times A \rightarrow R$ choisies de façon experte
- n paramètres/poids w_i appris (descente de gradient, apprentissage supervisé)

On stocke n poids ($n \ll S \times A$) et apprentissage conjoint des Q -valeurs (généralisation)

Fonctions d'approximation



Limites de l'approximation linéaire

- Difficile de choisir les bonnes caractéristiques expertes
- Limité par la linéarité : on ne peut représenter que des relations linéaires entre entrées et sorties

Solution : fonction d'approximation **non-linéaire** (réseau de neurones)

QLearning avec NN naïf

1 Rappels : Apprentissage par renforcement

2 Limites des approches classiques

3 Deep Q Learning

- QLearning avec NN naïf
- Experience replay
- Réseau cible
- Deep Q-Network (DQN) application

Approximation non-linéaire de la Q fonction

- Q_ω où ω : poids (appris) d'un réseau de neurones
- *Features* non-expertes : « apprises » par le réseau
- Entrée : version brute des perceptions



Action **Value** Approximation :

$$Q_\omega : S \times A \rightarrow \mathbb{R}$$



Action **Vector** Approximation :

$$Q_\omega : S \rightarrow \mathbb{R}^A$$

Approximation non-linéaire de la Q fonction

- Q_ω où ω : poids (appris) d'un réseau de neurones
- *Features* non-expertes : « apprises » par le réseau
- Entrée : version brute des perceptions



Action **Value** Approximation :
 $Q_\omega : S \times A \rightarrow \mathbb{R}$



Action **Vector** Approximation :
 $Q_\omega : S \rightarrow \mathbb{R}^A$

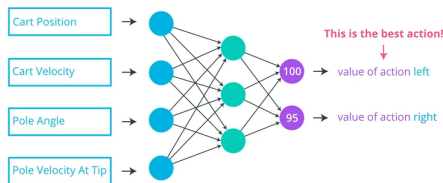
Autant de sortie que d'actions : cela permet de calculer plus facilement les $\max$ sur les actions.

Approximation non-linéaire de la Q fonction : exemple



State

	Min	Max
Cart Position	-2.4	2.4
Cart Velocity	-Inf	Inf
Pole Angle	$\sim -41.8^\circ$	$\sim 41.8^\circ$
Pole Velocity At Tip	-Inf	Inf



MLP avec 4 entrées et 2 sorties.

QL avec NN *naïf*

Fonction Q_ω paramétrée par les poids du NN ω .

Objectif : **minimiser l'erreur** sur les Q -valeurs (forme d'apprentissage supervisé) :

A chaque interaction (s, a, s', r) :

- prédiction de la Q -valeur $Q(s, a)$ (*forward pass*) : $\hat{y} = Q_\omega(s, a)$
- valeur cible de $Q(s, a)$: ? → **inconnue!!!** :

→ estimation par *amorçage* : $y = r + \gamma \max_{b \in A} \underbrace{Q_\omega(s', b)}_{\text{2nd forward pass}}$

QL avec NN naïf

Fonction Q_ω paramétrée par les poids du NN ω .

Objectif : **minimiser l'erreur** sur les Q -valeurs (forme d'apprentissage supervisé) :

A chaque interaction (s, a, s', r) :

- prédiction de la Q -valeur $Q(s, a)$ (*forward pass*) : $\hat{y} = Q_\omega(s, a)$
- **valeur cible de $Q(s, a)$: ? $\rightarrow$ inconnue !!!** :

$\rightarrow$ estimation par *amorçage* : $y = r + \gamma \max_{b \in A} \underbrace{Q_\omega(s', b)}_{\text{2nd forward pass}}$

A chaque interaction (s, a, s', r) : une itération de la descente de gradient (i.e. mise à jour des poids ω)

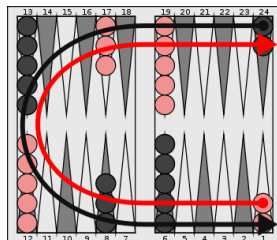
- fonction de perte MSE^a : $J_\omega = (y - \hat{y})^2$
- calcul du gradient de la fonction MSE par rapport à ω (**suppose ici que y est indépendant de ω !**) :

$$\nabla_\omega J_\omega = -2(y - \hat{y}) \nabla_\omega \hat{y}$$

- mise à jour des poids ω avec $\alpha \in [0, 1]$ coefficient d'apprentissage :

$$\begin{aligned} \omega &= \omega - \frac{\alpha}{2} \nabla_\omega J_\omega \\ &= \omega + \alpha (r + \gamma \max_{b \in A} q_\omega(s', b) - q_\omega(s, a)) \nabla_\omega q_\omega(s, a) \end{aligned}$$

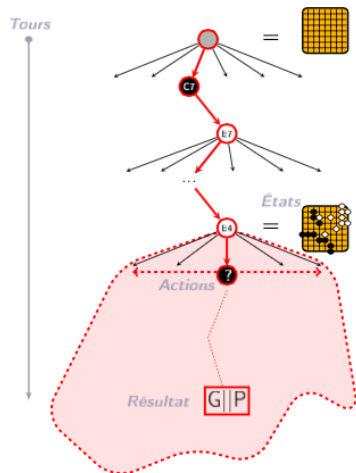
Application : TD-Gammon [Tesauro, 1992]



Backgammon

- jeu à 2 joueurs sur plateau, avec dés
- 2 ensembles de 15 dames
- joueur gagne lorsqu'il sort toutes ses dames du plateau (pour cela, il faut amener toutes ses dames dans le dernier quadrat)
- à chaque jet de dés, le joueur choisit parmi un grand nombre d'options pour déplacer ses dames (20 en moyenne)

Application : TD-Gammon [Tesauro, 1992]



Backgammon : Difficultés

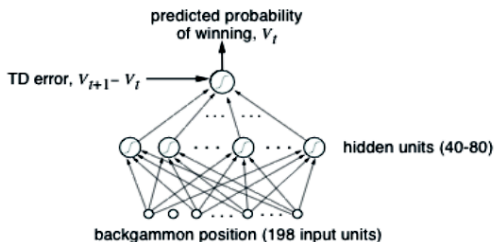
- Grand nombre de positions sur le plateau (30 pièces et 26 cases)
→ exclut les méthodes tabulaires
- Beaucoup d'options pour chaque jet de dés
- Pour considérer les mouvements futurs de l'adversaire, il faut considérer les jets de dés possibles.
→ Facteur de branchement d'environ 400

Application : TD-Gammon [Tesauro, 1992]

AR appliqué au Backgammon

- $TD(\lambda)$ avec retro propagation dans un réseau de neurones
- une couche d'entrée avec 198 entrées qui codent des informations sur la position des dames sur le plateau
- une couche cachée de 40 neurones
- une couche de sortie avec 3 valeurs pour estimer la probabilité de gagner de façon ordinaire, gammon ou backgammon.

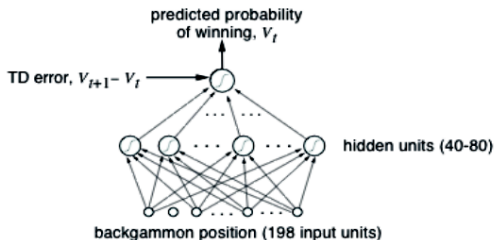
$V_t(s)$ utilisé pour estimer la probabilité de gagner depuis s .



TD-Gammon [Tesauro, 1992]

AR appliqué au Backgammon

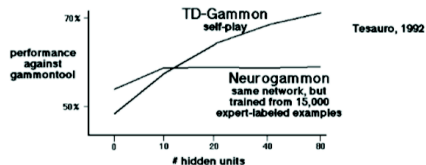
- récompenses immédiates : +100 si gagne, -100 si perd, 0 sinon
- après chaque jet de dés, le réseau évalue toutes les positions possibles depuis la position courante. Celle de plus forte valeur est choisie. La récompense reçue est alors rétro-propagée dans le réseau.
- Apprend en jouant contre soi-même 300000 parties
→ joue aussi bien que le meilleur joueur du monde



TD-Gammon [Tesauro, 1992]

Plusieurs versions

- Neurogammon : apprentissage supervisé sur corpus expert + informations caractéristiques en entrées
- TD Gammon 0.0 : apprentissage contre soi-même en mode TD
- TD Gammon 1.0 : apprentissage contre soi-même en mode TD + informations caractéristiques en entrées
- TD Gammon 2 : *2-ply search* : prise en compte des mouvements possibles de l'adversaire
- TD Gammon 3 : *3-ply search*



Program	Hidden Units	Training Games	Opponents	Results
TD-Gam 0.0	40	300,000	other programs	tied for best
TD-Gam 1.0	80	300,000	Robertie, Magriel, . . .	-13 points / 51 games
TD-Gam 2.0	40	800,000	various Grandmasters	-7 points / 38 games
TD-Gam 2.1	80	1,500,000	Robertie	-1 point / 40 games
TD-Gam 3.0	80	1,500,000	Kazaros	+6 points / 20 games

QL avec NN *naïf* : conclusion

Principe

- Apprentissage supervisé pour mise à jour des poids du NN qui approxime la Q fonction
- Amorçage pour les valeurs cibles

Instabilités

- corrélations entre interactions (s, a, s', r) envoyées successivement au NN
→ Experience replay
- corrélations entre valeur cible y et paramètres ω
→ Réseau cible

Experience replay

1 Rappels : Apprentissage par renforcement

2 Limites des approches classiques

3 Deep Q Learning

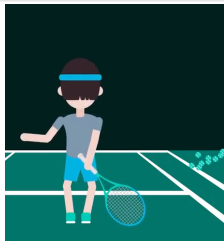
- QLearning avec NN naïf
- **Experience replay**
- Réseau cible
- Deep Q-Network (DQN) application

Problème : Corrélations entre interactions

Interactions (s, a, s', r) utilisées **dans ordre chronologique** pour la mise à jour des poids du NN :

- une action d'une interaction affecte les états des interactions suivantes.
- **difficulté** pour l'apprentissage supervisé : les données ne sont pas une réalisation de variables aléatoires *iid*^a

a. variables indépendantes et identiquement distribuées



Apprentissage "global" et corrélations → mise à jour d'un (s, a) peut entraîner des modifications incontrôlées d'autres (s, a)

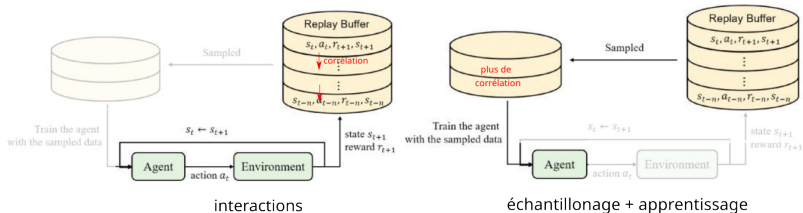
Risque de lenteur d'apprentissage ou de divergence.

Experience Replay : principe

Replay Buffer

2 phases entrelacées :

- Interaction avec l'environnement : mémorise dans un buffer les interactions rencontrées en suivant la politique courante (sans apprentissage)
- Échantillonnage + Apprentissage : tire **aléatoirement** un mini-batch d'interactions dans le buffer pour mettre à jour les poids ω



Deep Q-Learning avec *Experience Replay* : algorithme**Algorithm 1** DQN

```

1: Initialiser le replay buffer  $R$  de capacité  $N$ 
2: Initialiser la fonction  $Q$  avec des poids aléatoire  $\theta$ 
3: for episode  $e = 1$  à  $M$  do
4:   Récupérer l'état initial  $s_1$ 
5:   for étapes  $t = 1$  à  $T$  do
6:     Choisir  $a_t$  dans  $s_t$  selon une stratégie  $\epsilon$ -greedy
7:     Exécuter  $a_t$  et récupérer  $s_{t+1}$  et  $r_t$ 
8:     Ajouter la transition  $(s_t, a_t, r_t, s_{t+1})$  dans  $R$ 
9:      $s_t \leftarrow s_{t+1}$ 
10:    if  $t \% \text{learnstep} == 0$  then
11:      Récupérer un minibatch de taille  $d$  dans  $R$ :  $(s_j, a_j, r_j, s_{j+1})$ 
12:      Calculer la prédiction  $y_j = Q_\theta(s_j, a_j)$ 
13:      Calculer la cible  $y_j = \begin{cases} r_j & \text{si } e \text{ se termine} \\ r_j + \gamma \max_b Q_\theta(s_{t+1}, b) & \text{sinon} \end{cases}$ 
14:      Mettre à jour  $\theta$ : effectuer une étape de descente de gradient pour
        minimiser  $(y_j - \hat{y}_j)^2$  (MSE loss)
15:    end if
16:  end for
17: end for

```

phase d'interaction

mémorisation dans buffer

échantillonnage

phase d'apprentissage

Experience Replay : intérêts

- 1 réduit la corrélation entre interactions successives.
- 2 accélère l'apprentissage : une interaction est potentiellement utilisée dans plusieurs mises à jour des poids.
- 3 réduction de l'oubli (*catastrophic forgetting*) : réutilise des interactions rares ou anciennes.

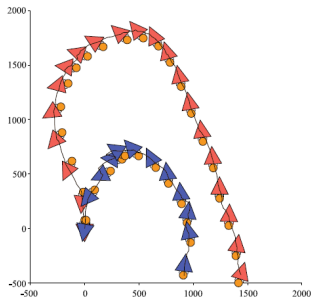


By learning how to play on water level, our agent will forget how to behave on the first level

Applications de l'Experience Replay : Robocup soccer

Neural Fitted Q-learning [Stone et al., Riedmiller et al., 2005]

- Robot footballeur qui apprend différentes techniques pour jouer au foot
- Deep Q-learning avec experience replay : MLP avec 9 entrées (6 pour l'état, 3 pour les actions), 2 couches cachées de 20 neurones chacune, un neurone de sortie.
- Phase d'apprentissage d'environ 1h30.



Technique de *dribble* : *hand-coded* (rouge) vs. neural FQ (bleu)

Robocup soccer

Real World Robot Learning

Learning to Dribble by
Success and Failure



Machine
Learning



Prof. Dr. Martin Riedmiller
Department of Computer Science
Albert-Ludwigs-University Freiburg



UNI
FREIBURG

Robocup soccer

	'00	'01	'02	'03	'04	'05	'06	'07	'08
Simulation League									
NeuroKick	•	•	•	•	•	•	•	•	•
NeuroIntercept	•	•	•	•		○	○		
NeuroGo2Pos	•	•	•	•	•				
NeuroADB								•	•
NeuroAttack	○	•	•	•	•		•	•	•
NeuroPenalty				•	•	•	•	•	•
Rank	2	2	3	3	2	1	2	1	1
MidSize League									
NeuroMotorSpeed								○	○
NeuroGo2Pos							○	○	○
LmapIntercept							•	•	•
NeuroDribble								•	•
Rank							1	1	3

Techniques apprises dans l'équipe Brainstormers Tribots

Réseau cible

1 Rappels : Apprentissage par renforcement

2 Limites des approches classiques

3 Deep Q Learning

- QLearning avec NN naïf
- Experience replay
- **Réseau cible**
- Deep Q-Network (DQN) application

Problème

Corrélation entre valeur cible y et paramètres ω

- La valeur cible pour app. supervisé, estimée par amorçage, **dépend de ω** :

$$y = r + \gamma \max_{b \in A} \underbrace{Q_{\omega}(s', b)}_{\text{2nd forward pass}}$$

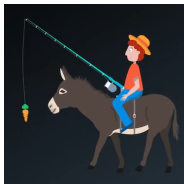
- Mais le calcul du gradient de l'erreur par rapport à ω **suppose que y est indépendant de ω** ! :

$$\nabla_{\omega} J_{\omega} = -2(y - \hat{y}) \nabla_{\omega} \hat{y}$$

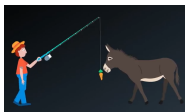
- On réalise une mise à jour des poids ω avec une **valeur cible y non-stationnaire** :

$$\omega = \omega + \alpha \underbrace{\left(r + \gamma \max_{b \in A} q_{\omega}(s', b) \right)}_y - \underbrace{q_{\omega}(s, a)}_{\hat{y}} \nabla_{\omega} q_{\omega}(s, a)$$

→ instabilité dans l'apprentissage



Réseau cible : principe



Figurer la cible

Découpler les paramètres utilisés pour calculer la cible ω^- et ceux mis à jour ω :

$$\omega = \omega + \alpha \underbrace{(r + \gamma \max_{b \in A} q_{\omega^-}(s', b))}_{\hat{y}} - \underbrace{q_{\omega}(s, a)}_{\hat{y}} \nabla_{\omega} q_{\omega}(s, a)$$

- utilisation d'un second NN cible, de poids ω^- , pour calculer la valeur cible
- mise à jour à chaque phase d'entraînement des poids ω (descente de gradient)
- mise à jour des poids cible ω^- avec les poids ω selon une fréquence plus faible

Intérêts

- stabilisation de la cible
- réduction des corrélations entre cible et paramètres changeants

Deep Q-Network (DQN) : algorithme (2013)

Deep Q-Learning avec expérience replay et réseau cible

Algorithm 1 DQN avec réseau cible

```

1: Initialiser le replay buffer  $R$  de capacité  $N$ 
2: Initialiser la fonction  $Q$  avec des poids aléatoire  $\theta$ 
3: Initialiser la fonction cible  $\hat{Q}$  avec des poids  $\theta^- = \theta$ 
4: for episode  $e = 1$  à  $M$  do
5:   Récupérer l'état initial  $s_1$ 
6:   for étapes  $t = 1$  à  $T$  do
7:     Choisir  $a_t$  dans  $s_t$  selon une stratégie  $\epsilon$ -greedy
8:     Exécuter  $a_t$  et récupérer  $s_{t+1}$  et  $r_t$ 
9:     Ajouter la transition  $(s_t, a_t, r_t, s_{t+1})$  dans  $R$ 
10:     $s_t \leftarrow s_{t+1}$ 
11:    if  $t \% \text{learnstep} == 0$  then
12:      Récupérer un minibatch de taille  $d$  dans  $R$ :  $(s_j, a_j, r_j, s_{j+1})$ 
13:      Calculer la prédiction  $\hat{y}_j = Q_\theta(s_j, a_j)$ 
14:      Calculer la cible  $y_j = \begin{cases} r_j & \text{si } e \text{ se termine} \\ r_j + \gamma \max_b \hat{Q}_{\theta^-}(s_{t+1}, b) & \text{sinon} \end{cases}$ 
15:      Mettre à jour  $\theta$ : effectuer une étape de descente de gradient pour
        minimiser  $(y_j - \hat{y}_j)^2$  (MSE loss)
16:      Mettre à jour la cible  $\hat{Q}$  tous les  $C$  pas ou en soft update
17:    end if
18:  end for
19: end for

```

phase d'interaction

phase d'apprentissage

Deep Q-Network (DQN) application

1 Rappels : Apprentissage par renforcement

2 Limites des approches classiques

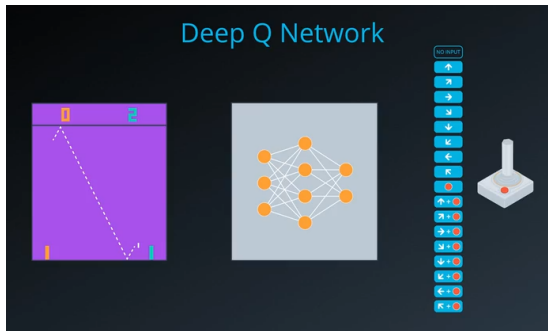
3 Deep Q Learning

- QLearning avec NN naïf
- Experience replay
- Réseau cible
- Deep Q-Network (DQN) application

Application DQN : jeux Atari



video

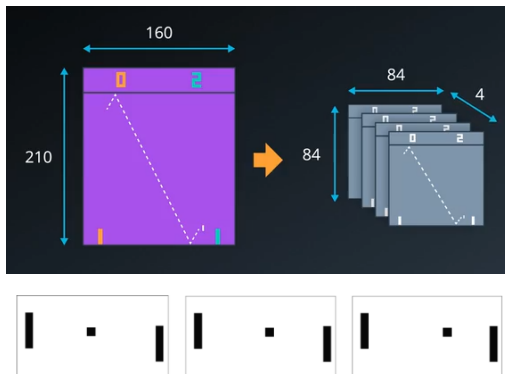


- entrées : image
- sorties : Q -valeurs de chaque action discrète (4 à 18)
- récompense : changement de score à chaque étape
- experience replay : stockages des 100 000 dernières interactions
- réseau cible

Playing Atari with Deep Reinforcement Learning, V. Mnih et. al, 2013

Human-level control through deep reinforcement learning, V. Mnih et. al, Nature, 2015

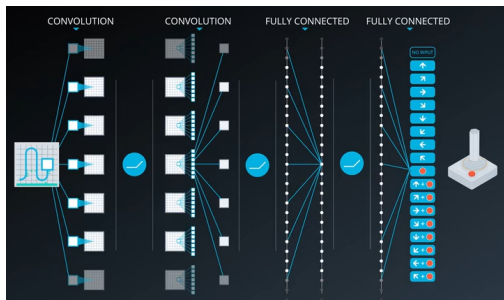
Application DQN : jeux Atari



Traitement des images en entrée $S = 84 \times 84 \times 4$

- réduction des images en niveau de gris de taille 84×84 pour réduire le temps de calcul de l'apprentissage sans perte d'information
- agrégation (*stack*) des 4 dernières images : réduit le problème de limitation temporelle, donne à l'agent une information sur les mouvements : direction, vitesse, etc.

DQN pour les jeux Atari



Architecture du NN

- 2 CNNs avec ReLU pour extraire les relations spatiales et temporelles entre les 4 images
- 1 couche fully-connected avec ReLU
- 1 couche fully-connected de sortie produit le vecteur des valeurs d'action
- même architecture pour tous les jeux Atari

DQN : Conclusion

- fonction d'approximation non-linéaire pour fonction Q , avec autant de sorties du NN que d'actions
- apprentissage supervisée avec cible mouvante
- *experience replay* pour réduire la corrélation entre interactions successives dans un même épisode
- réseau cible (*target network*) pour réduire les corrélations dues à la cible y mouvante

Limitation principale de DQN : impossible d'avoir des actions continues ...